

Benchmark Harness (BENCH_HARNESS)

Version: 2.0.1 | Updated: 2026-06-27

Lives at `benchmarks/docs/` (renamed from `docs/HARNESS.md`). Also ingested into the help corpus, so you can ask AIStudio how to run benchmarks. Companion: the audited evidence in `BENCH - Canonical Suite - README and Synthesis` (same folder), and how-to-read-a-benchmark in TUTORIAL §5.

AIStudio ships with a benchmark harness (`benchmarks/bench.py`) that runs a structured question set against any corpus and produces timestamped reports. It is designed for the demo corpus out of the box but works with any corpus and any YAML question file you provide.

Reports are written to `benchmarks/<corpus>/reports/` as paired `.md` and `.json` files, named by corpus and timestamp.

Quick Start (Demo Corpus)

Make sure the backend is running (`ais_start`), then:

```
ais_bench
```

This runs all 14 demo questions against the `demo` corpus and writes a timestamped report to `benchmarks/demo/reports/`.

CLI Reference

```
ais_bench [OPTIONS]
```

Options:

<code>--corpus NAME</code>	Corpus to query (default: demo)
<code>--scope NAME</code>	Restrict to a named corpus subset (e.g. lang_en) – reads data/corpora/<corpus>/scopes/<corpus>_<scope>_scope.yaml
<code>--questions STEM</code>	Named question subset by STEM (not a path). Empty = the flat

```

--batch          default set; a STEM resolves to questions/<corpus>_<stem>_questions.
                An unknown STEM is a HARD ERROR (no silent fallback).
--top-k INT      Run the pinned canonical suite from benchmarks/batch/bench_canonical
                (multiple corpus/scope/question combinations in one invocation).
--temperature FLOAT LLM sampling temperature 0.0-2.0 (default: 0.3)
--model NAME     a corpus with no stored default falls back to the system default of
                Ollama model name (default: API default)
--api URL        Backend URL (default: http://localhost:8000)
--no-markdown    Skip writing .md report
--full          Include full answers in report (default: first 4 paragraphs)
--fit-policy MODE What to do when a model won't fit this machine's memory (skip | down
                force). Omitted + a model that won't fit → bench asks [YES/n] before
                (needs a terminal; a fully non-interactive run fails closed). (AISTU
--dry-run        Without --batch, print the resolved run set and exit WITHOUT executing
                Without --batch it stops with a notice rather than running a single
--help          Show this help message
--version       Show version

(`--canonical` is kept as a silent deprecated alias of `--batch`.)

```

Question sets — one home. The default set is the flat file `benchmarks/<corpus>/<corpus>_questions.yaml` (used when `--questions` is omitted). Named subsets live one level down in `benchmarks/<corpus>/questions/` as `<corpus>_<stem>_questions.yaml`, selected by STEM: `ais_bench --corpus sec_10k --questions frontier` resolves to `benchmarks/sec_10k/questions/sec_10k_frontier_questions.yaml`. A STEM that doesn't resolve is a hard error — the harness will not silently fall back to the default.

Question File Format

Questions are YAML files organized by topic. Each question has an `id`, a `question` string, optional `keywords` (a soft signal feeding the GREEN/AMBER/RED rating — see Scoring below, not a binary gate), and optional `notes`:

```

- topic: Architecture Methodology
  questions:
    - id: my_first_question
      question: What is the relationship between business strategy and technology strategy?
      keywords: [business, strategy, technology]
      notes: Core intellectual thread — business strategy drives technology strategy.

    - id: my_second_question
      question: How do you design an IT organization around architectural principles?
      keywords: [organization, architecture, principles]

```

Place question files at `benchmarks/{corpus}/{corpus}_questions.yaml` for auto-detection. See `benchmarks/demo/demo_questions.yaml` for a fully annotated example.

Bringing Your Own Corpus

1. Create and ingest your corpus

Create a corpus via the UI, upload your documents, and ingest them.

2. Write a question file

Create `benchmarks/{corpus}/{corpus}_questions.yaml` following the format above.

3. Run the benchmark

```
ais_bench --corpus my_corpus
```

Reports are written to `benchmarks/my_corpus/reports/`.

Running Systematic Benchmarks

The canonical suite (`--batch`)

The pinned, reproducible benchmark set lives at `benchmarks/batch/bench_canonical.yaml` and runs with:

```
ais_bench --batch
```

It executes the canonical runs — across both shipped corpora, at the canonical depth — in one invocation, writing each run's report to its corpus `reports/`. This is how the headline numbers in `BENCH - Canonical Suite - README` and `Synthesis` are regenerated. Use it to reproduce the published evidence; use the individual flags below to explore.

Compare models

```
ais_bench --model gemma3:27b    # the SEC 10-K benchmark model
ais_bench --model llama3.1:8b
ais_bench --model llama3.1:70b
```

Compare retrieval depth

```
ais_bench --top-k 3
ais_bench --top-k 5
ais_bench --top-k 10
```

Compare temperature

```
ais_bench --temperature 0.0
ais_bench --temperature 0.3
ais_bench --temperature 0.7
```

Each run produces a separate timestamped report in `benchmarks/demo/reports/`:

```
benchmarks/demo/reports/benchmark_demo_2026-04-15_21-30.md
benchmarks/demo/reports/benchmark_demo_2026-04-15_21-30.json
```

Reading a Report

Each report contains:

Configuration — full snapshot (corpus, model, top-k, temperature, timestamp) so every report is self-describing and reproducible.

Summary — total questions, the  rating tally, binary pass rate (back-compatible), average latency.

Infrastructure — vector store, embedding model, reranker, corpus stats.

Results table — per question: latency, rating () , citation density, sources, notes.

Detailed results — full answer text with inline citations, per question.

The first query in a cold session is slow (20–50s) while the LLM loads into memory. Subsequent queries run at ~6–7s warm on M4 Pro/Max, or ~26–28s on M4 Air — latency is hardware-bound, not corpus-bound. The benchmark summary reflects warm latency once the first query completes.

Scoring — the GREEN/AMBER/RED rating

Since AIStudio_878, `evaluate()` returns a tri-state `rating ∈ {GREEN, AMBER, RED}` as the primary verdict, with the older binary `pass` demoted to a back-compat soft signal:

-  **GREEN** — cited, right firm, honest, keywords present, adequate citation density.
-  **AMBER** — cited and plausibly right, but with a soft weakness (a keyword miss, low citation density, or partial entity coverage) → read up / audit.
-  **RED** — no citations, honest-empty, or wrong firm (entity coverage 0 with a filter active).

Keyword matching is demoted from the verdict to one input among several — a GREEN answer can miss a keyword and an AMBER one can hit them all. The thresholds are v1 defaults, calibrated against the audited canonical runs; the rating is a triage signal, not a grade. The contract is still the audit: verify the cited chunk, never the colour alone.

SEC 10-K Corpus

To benchmark against the full 101-filing SEC corpus (21 firms):

```
# Download first (one-time, ~5 min); then ingest via the UI Upload button
ais_download_sec_10k

# Then benchmark
ais_bench --corpus sec_10k          # K=10 is the stored default for sec_10k
```

Default question set ships at `benchmarks/sec_10k/sec_10k_questions.yaml` ; named subsets (e.g. `frontier`) live in `benchmarks/sec_10k/questions/` .

Reports are written to `benchmarks/sec_10k/reports/` .

What's Coming

- **Cross-run comparison** — diff two reports on latency, answer quality, source overlap (partially delivered via `--batch` + the canonical suite)
- **Hardware metadata** — machine specs recorded in report header for M4 Air vs M4 Max comparisons
- **Model quality evaluation** — human eval framework for 8b vs 70b answer quality

See `docs/PRODUCT_ROADMAP.md` for the full roadmap.

*** **